

20.1-3: Describe efficient algorithms for computing G^T from G , for both the adjacency-list and adjacency-matrix representations of G . Analyze the running times of your algorithms.

Pseudocode for transpose of graph using adjacency-list:

findTranspose(G : graph)

Let T be a graph that will become the transpose of G
for u in $G.V$ // u is a vertex

for e in $G.Adj[u]$ // e is a vertex

Add($T.Adj[e], u$) // Add function appends " u " onto
// the end of the linked list
// for vertex e

return T

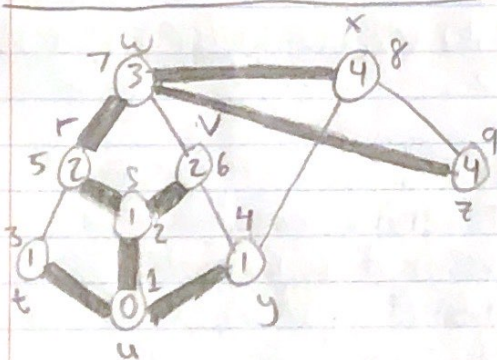
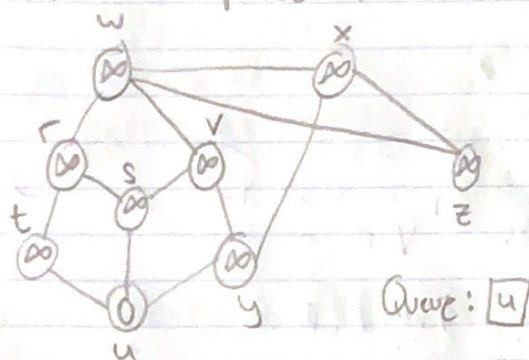
Basically, the code above would create a graph called T that has the same attributes as graph G , except the adjacency list isn't filled, and the edges are flipped. Then, the for-loops traverse the adjacency list of G . Every vertex " u " is added into the adjacency list for " e ". For example:

$G:$	$\begin{matrix} ① \rightarrow ② \rightarrow ④ \\ ② \rightarrow ① \rightarrow ④ \\ ③ \rightarrow ② \\ ④ \rightarrow ① \rightarrow ③ \end{matrix}$	$\rightarrow$ becomes $\rightarrow$	$\begin{matrix} G^T: ① \rightarrow ② \rightarrow ④ \\ ② \rightarrow ① \rightarrow ③ \\ ③ \rightarrow ④ \\ ④ \rightarrow ① \rightarrow ② \end{matrix}$
------	--	-------------------------------------	---

The runtime for this algorithm is $O(|V| + |E|)$, since all vertices and edges will be traversed.

If an adjacency matrix is being used to represent the graph, then you can simply take the transpose of the matrix: the elements of row 1 become the elements of column 1, then do the same for row 2 and column 2, until you've done them all. Since there are $|V| \times |V|$ elements in an adjacency matrix, you'd have to traverse $|V| \times |V|$ elements. So, the runtime for this algorithm is $O(|V|^2)$.

20.2-2: Show d and π values that result from running BFS on the undirected graph of Figure 20.3 using vertex "u" as the source. Assume that neighbors are visited in alphabetical order.



	u	s	t	y	r	v	w	x	z
d	0	1	1	1	2	2	3	4	4
π	NIL	u	u	u	s	s	r	w	w

All d -values and π -values for each vertex

20.2-4: What is the running time of BFS if we represent its input graph by an adjacency matrix and modify the algorithm to handle this form of input?

BFS on an adjacency list is $O(M + |V|)$, since we would traverse every vertex and all of the edges too.

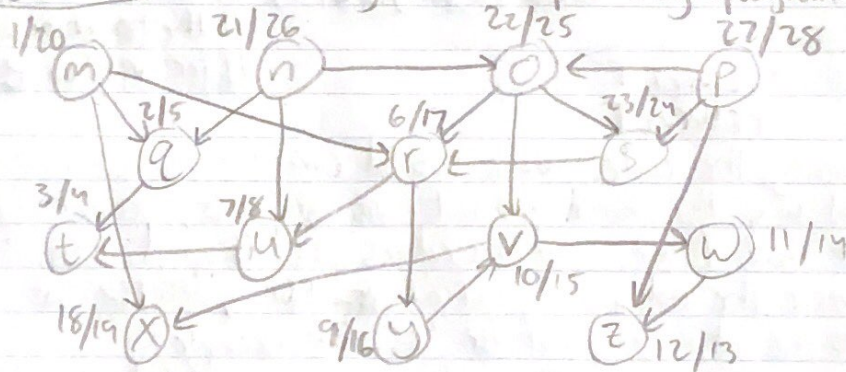
(NEXT PAGE)

20.2-4 (cont.):

If BFS was done on an adjacency matrix, then we would have to traverse every element of it. Since an adjacency matrix has a size of $|V| \times |V|$, then we'd have to traverse $|V| \times |V|$ elements.

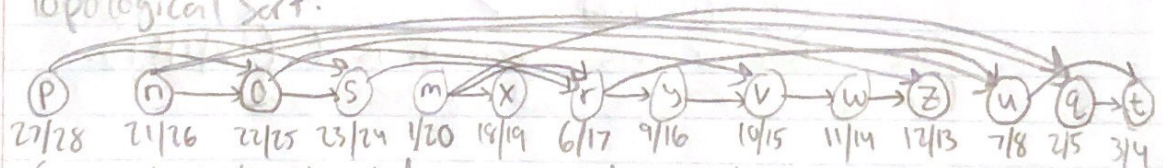
So, the running time of BFS if we represent its input graph by an adjacency matrix and modify the algorithm accordingly would be $O(|V|^2)$.

20.4-1: Show ordering of vertices produced by Topological sort on this dag



(DFS is done in alphabetical order)

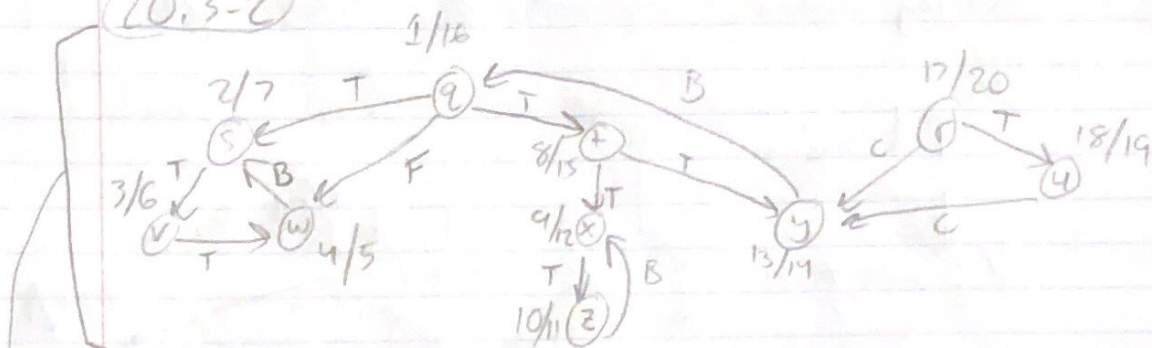
Topological Sort:



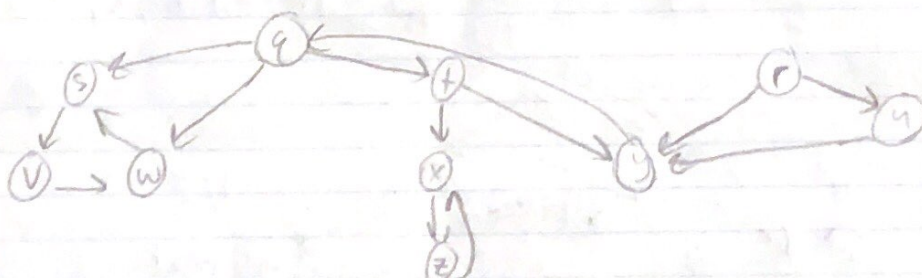
(Sorry about the directed lines, I didn't realize that it'd become this messy with 14 vertices).

Parth Patel

20.3-2



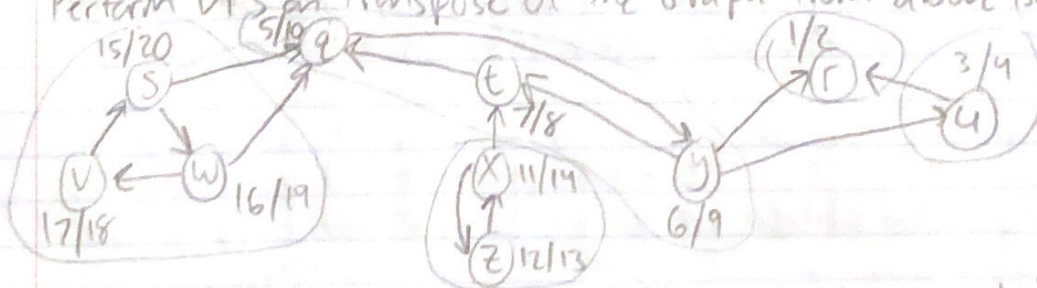
20.5-2



We will use the finishing times from 20.3-2, since it's the same DFS on the same graph:

Decreasing finishing times: r, u, q, t, y, x, z, s, v, w

Perform DFS on Transpose of the Graph from above (start "r"):



Above are all the separate components that are strongly connected components.

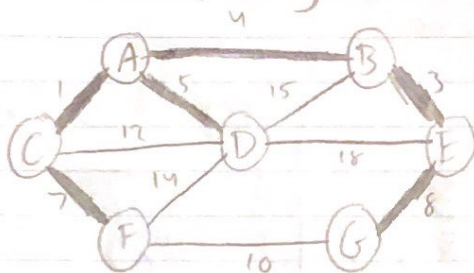
The strongly connected components of this graph are:

$SCC = \{\{q, t, y\}, \{r\}, \{u\}, \{s, v, w\}, \{x, z\}\}$

Parth Patel
CSCI 4104

10/10/23
Problem Set 2 (cont.)

Minimum Spanning Tree Problem



This example has a unique MST, which is shown by the thicker lines

- (a) List the edges that are in MST in the order that Kruskal's Algorithm would find them in.
Edges in order: $\{(A,C), (B,E), (A,B), (A,D), (C,F), (E,G)\}$
Corresponding weights: 1, 3, 4, 5, 7, 8
Total weight: 28
The edges with weights 10, 12, 14, 15, 18 are skipped at the end, since they're in same tree.
- (b) List the edges that are in MST in the order that Prim's algorithm would find them in, using vertex A as the root.
Edges in order: $\{(A,C), (A,B), (B,E), (A,D), (C,F), (E,G)\}$
Corresponding weights: 1, 4, 3, 5, 7, 8
Total weight: 28
The edges with weights 10, 12, 14, 15, 18 are skipped at the end, since they lead to other vertices that are already in the same tree.
- (c) List the edges that are in MST in the order that Prim's algorithm would find them in, using vertex G as the root.
Edges in order: $\{(G,E), (E,B), (B,A), (A,C), (A,D), (C,F)\}$
Corresponding weights: 8, 3, 4, 1, 5, 7
Total weight: 28
The edges with weights 10, 12, 14, 15, 18 are skipped at the end, since they lead to other vertices that are already in the same tree.